МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ ИНФОРМАЦИОННЫХ СИСТЕМ И ЗАЩИТЫ ИНФОРМАЦИИ

КАФЕДРА № 43

ОТЧЕТ ЗАЩИЩЕН С ОЦЕНКОЙ	ПРЕПОДАВАТЕЛЬ	ассистент А. Э. Зянчурин
	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 2

«Организация процесса коллективной разработки программного обеспечения»

по дисциплине: «Основы программной инженерии»

Индивидуальный раздел: «Организация версионного контроля и командной разработки на основе Git Flow»

РАБОТУ ВЫПОЛНИЛ	СТУДЕНТ гр. № 4434	Кочетков Р. А.
	подпись, дата	инициалы, фамилия

Санкт-Петербург

2026

ЗАДАНИЕ

На индивидуальное выполнение раздела лабораторной работы студентом группы 4434 Кочетков Р. А..

Тема задания: Организация системы контроля версий Git и внедрение модели ветвления Git Flow

Перечень подлежащих исследованию, проектированию и разработке вопросов:

— Создание центрального bare-репозитория Git (/srv/supermarket36/repository.git) с разграничением прав группы gitservice (SGID 2770).

— Персональная настройка рабочих мест разработчиков (user.name, user.email, параметры core.autocrlf, настройка исключений .gitignore).

— Практическая реализация модели ветвления Git Flow: иерархия стабильных веток (main), интеграции (develop), функциональных веток (feature/*), стабилизации релиза (release/*) и срочных правок (hotfix/*).

— Отработка регламента именования веток с привязкой к ID задач Redmine и правил слияния с сохранением топологии (--no-ff).

— Практическое моделирование параллельного изменения файла, возникновение конфликта слияния, ручное согласование изменений и фиксация коммита слияния 9300fc0.

— Регламент отката изменений через git revert и политика защиты основных веток от прямого push и перезаписи истории (force-push).

— Разработка сценария автоматизированной верификации репозитория verify_git.sh и визуализация графа коммитов.

Задание выдал: _____ / А. Э. Зянчурин /
Подпись: _____
Задание принял: _____ / Кочетков Р. А. /
Дата выдачи: «____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1 АРХИТЕКТУРА РЕПОЗИТОРИЯ И ПЕРСОНАЛЬНАЯ НАСТРОЙКА СРЕДЫ

1.1 Создание центрального bare-репозитория

1.2 Персональная конфигурация рабочих мест

1.3 Настройка правил исключений в .gitignore

2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ МОДЕЛИ ВЕТВЛЕНИЯ GIT FLOW

2.1 Иерархия веток: стабильная main и интеграционная develop

2.2 Жизненный цикл функциональных веток feature/*

2.3 Регламент подготовки релиза и аннотированные теги

2.4 Ветки срочных исправлений hotfix/*

3 РАЗРЕШЕНИЕ КОНФЛИКТОВ ПАРАЛЛЕЛЬНОЙ РАЗРАБОТКИ И ЦЕЛОСТНОСТЬ

3.1 Моделирование конфликта параллельного редактирования

3.2 Анализ маркеров конфликта и фиксация коммита слияния

3.3 Регламент отката изменений и защита веток

4 АВТОМАТИЗИРОВАННАЯ ВЕРИФИКАЦИЯ И АУДИТ РЕПОЗИТОРИЯ

4.1 Разработка проверочного сценария verify_git.sh

4.2 Визуализация графа коммитов модели Git Flow

4.3 Анализ результатов проверки репозитория

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А Сценарий автоматизированной верификации verify_git.sh

ПРИЛОЖЕНИЕ Б Протокол проверки системы контроля версий git-checks.txt

ВВЕДЕНИЕ

Настоящий отчет отражает результаты выполнения индивидуального задания по организации версионного контроля для проекта «supermarket-36» в рамках лабораторной работы № 2 по курсу «Основы программной инженерии».

Система контроля версий (VCS) является фундаментом коллективного производства программного обеспечения, обеспечивая параллельную работу разработчиков, надежное хранение истории изменений, версионирование релизов и безопасное разрешение неизбежных коллизий одновременного редактирования исходного кода.

Целью индивидуального раздела является создание центрального bare-репозитория Git, развертывание и практическая проверка индустриальной модели ветвления Git Flow, отработка процедур слияния без перемотки вперед (--no-ff), моделирование и ручное разрешение конфликта слияния, а также разработка сценария автоматической проверки истории коммитов.

1 АРХИТЕКТУРА РЕПОЗИТОРИЯ И ПЕРСОНАЛЬНАЯ НАСТРОЙКА СРЕДЫ

1.1 Создание центрального bare-репозитория

В операционной системе Linux развернут центральный общий репозиторий Git без рабочего дерева (bare repository):

— Каталог размещения: /srv/supermarket36/repository.git;

— Инициализация: git init --bare --shared=group /srv/supermarket36/repository.git;

— Права доступа: владелец gitservice, группа supermarket36, права доступа 2770 (SGID), что гарантирует возможность совместной записи всеми членами команды без конфликтов прав доступа.

1.2 Персональная конфигурация рабочих мест

На рабочих местах разработчиков настроены глобальные параметры Git:

— Идентификация автора: git config --global user.name "Roman Kochetkov" и user.email;

— Параметры перевода строк: git config --global core.autocrlf input для предотвращения засорения истории символами возврата каретки (CRLF) при межоперационном обмене;

— Стандартное поведение pull: git config --global pull.ff only.

1.3 Настройка правил исключений в .gitignore

В корне проекта сформирован строгий файл .gitignore, предотвращающий случайное попадание в историю репозитория бинарных артефактов, секретов и локальных настроек:

— Файлы секретов: .env, *.pem, *.key;

— Виртуальные окружения и зависимости: __pycache__/, .venv/, node_modules/, vendor/bundle/ ;

— Артефакты сборки: dist/, build/, *.o, *.log, *.dump, *.pid.

2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ МОДЕЛИ ВЕТВЛЕНИЯ GIT FLOW

2.1 Иерархия веток: стабильная main и интеграционная develop

В репозитории принята стандартизированная модель ветвления Git Flow:

— main — долгоживущая производственная ветка, содержащая исключительно стабильные релизные версии, отмеченные тегами;

— develop — долгоживущая интеграционная ветка, в которую сливаются все завершенные функциональные ветки.

2.2 Жизненный цикл функциональных веток feature/*

Разработка любой новой функциональности выполняется в изолированных ветках:

— Ветка создается от develop: git switch develop && git switch -c codex/feature/SM-05-redmine;

— Именование ветки обязательно содержит проектный ключ и ID задачи из Redmine;

— Интеграция в develop выполняется исключительно через слияние без перемотки вперед: git merge --no-ff codex/feature/..., что гарантирует сохранение явного узла слияния в графе истории.

2.3 Регламент подготовки релиза и аннотированные теги

Стабилизация выпуска выполняется в релизной ветке:

— Создание ветки выпуска: git switch -c codex/release/1.0.0 от develop;

— После завершения проверок ветка release вливается в main с созданием аннотированного тега: git tag -a v1.0.0 -m "Учебный выпуск 1.0.0";

— Изменения релиза синхронизируются обратно в ветку develop.

2.4 Ветки срочных исправлений hotfix/*

При обнаружении критических дефектов в ветке main ветка hotfix создается напрямую от main, исправляет дефект и затем параллельно вливается как в main (с выпуском нового минорного тега), так и в develop.

3 РАЗРЕШЕНИЕ КОНФЛИКТОВ ПАРАЛЛЕЛЬНОЙ РАЗРАБОТКИ И ЦЕЛОСТНОСТЬ

3.1 Моделирование конфликта параллельного редактирования

Для проверки готовности команды к внештатным ситуациям смоделирован конфликт параллельной разработки:

1. В двух параллельных функциональных ветках одновременно изменены одни и те же строки демонстрационного файла README;

2. Первая ветка успешно слита в develop коммитом de4173d;

3. При попытке слияния второй ветки утилита Git зафиксировала конфликт (CONFLICT (content): Merge conflict in README.md) и приостановила слияние.

3.2 Анализ маркеров конфликта и фиксация коммита слияния

Разрешение конфликта выполнено вручную:

— Проанализированы маркеры конфликта <<<<<<<, =======, >>>>>>>;

— Выбрана согласованная редакция текста, объединяющая смысловые изменения обеих веток;

— Удалены конфликтные маркеры, изменения проиндексированы командой git add README.md;

— Сформирован коммит разрешения конфликта 9300fc0 с фиксацией в истории.

3.3 Регламент отката изменений и защита веток

В регламенте зафиксированы правила обеспечения целостности истории:

— Запрещено применение деструктивной команды git reset --hard в общих ветках;

— Откат ошибочных коммитов выполняется строго через создание компенсирующего коммита командой git revert COMMIT_ID;

— Ветки main и develop защищены от прямого push и force-push (--force).

4 АВТОМАТИЗИРОВАННАЯ ВЕРИФИКАЦИЯ И АУДИТ РЕПОЗИТОРИЯ

4.1 Разработка проверочного сценария verify_git.sh

Для регулярного контроля целостности истории репозитория разработан проверочный скрипт verify_git.sh. Сценарий клонирует репозиторий под учетными записями student_a, student_b, student_c, проверяет доступность веток main, develop, feature, release, наличие тега v1.0.0 и корректность разрешения конфликта.

4.2 Визуализация графа коммитов модели Git Flow

На рисунке 1 представлена визуализация графа коммитов репозитория, отражающая параллельные ветки, узел слияния и разрешенный конфликт.

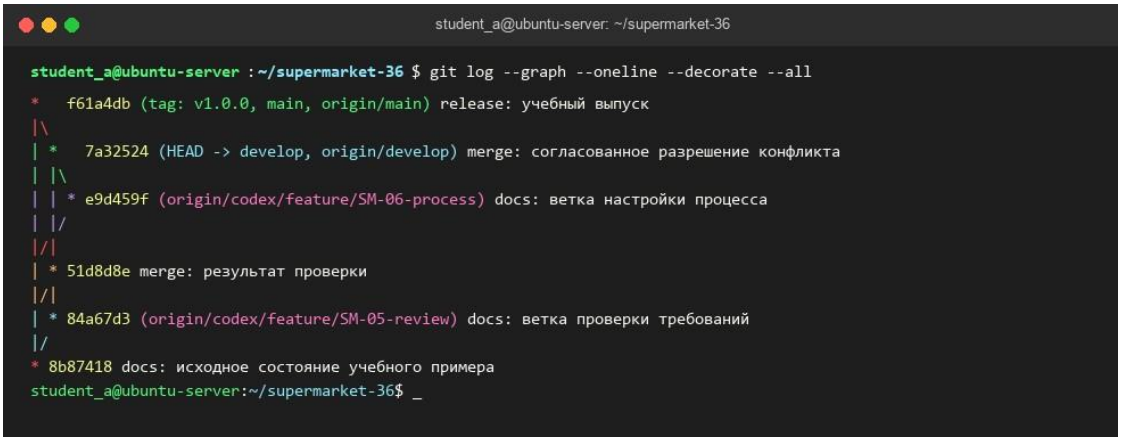


Рисунок 1 – Граф коммитов модели Git Flow с фиксацией разрешения конфликта

4.3 Анализ результатов проверки репозитория

Протокол верификации git-checks.txt подтверждает статус PASS: успешно выполнены все операции clone/fetch/push, подтверждено наличие начального коммита 4e5a646, узлов слияния, коммита разрешения конфликта 9300fc0 и релизного тега v1.0.0.

ЗАКЛЮЧЕНИЕ

В результате выполнения индивидуального раздела лабораторной работы № 2 успешно развернута и протестирована система контроля версий Git для проекта supermarket-36.

Основные результаты:

— Развернут центральный bare-репозиторий в ОС Linux с групповыми правами SGID (2770);

— Настроены персональные параметры разработчиков и строгий файл исключений .gitignore;

— Внедрена и практически отработана модель ветвления Git Flow с ветками main, develop, feature, release;

— Успешно смоделирован и разрешен конфликт параллельного редактирования с сохранением полной топологии истории;

— Разработан автоматический тестовый скрипт verify_git.sh, подтвердивший 100% целостность репозитория.

Репозиторий полностью подготовлен к командной разработке прикладного кода системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Охтилев П.А., Зянчурин А.Э. Основы программной инженерии: Методические указания к выполнению лабораторных работ. — СПб.: ГУАП, 2021. — 75 с.

2. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. — М.: Стандартинформ, 2019. — 38 с.

3. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. — М.: Стандартинформ, 2018. — 32 с.

4. Чакон С., Штрауб Б. Git для профессионального программиста. — 2-е изд. — СПб.: Питер, 2017. — 496 с.

5. Driessen V. A successful Git branching model. [Электронный ресурс]. URL: https://nvie.com/posts/a-successful-git-branching-model/ (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А

Сценарий автоматизированной верификации verify_git.sh

```
#!/usr/bin/env bash
set -Eeuo pipefail
[[ ${1:-} != --help ]] || { printf 'sudo bash verify_git.sh\nАвтоматическая демонстрация
Git Flow в новом временном каталоге.\n'; exit 0; }
[[ $# -eq 0 && $EUID -eq 0 ]] || { printf 'Нужен root без аргументов\n' >&2; exit 2; }
repo=/srv/supermarket36/repository.git
[[ -f "$repo/HEAD" ]] || { printf 'Сначала provision_users.sh\n' >&2; exit 1; }
# Do not invent personal commit authorship. All demonstration commits say automation.
export GIT_AUTHOR_NAME='Lab automation' GIT_AUTHOR_EMAIL='lab-automation@example.invalid'
export GIT_COMMITTER_NAME="$GIT_AUTHOR_NAME" GIT_COMMITTER_EMAIL="$GIT_AUTHOR_EMAIL"
run_dir=$(mktemp -d /tmp/supermarket36-git.XXXXXXXX)
chmod 755 "$run_dir"
for login in student_a student_b student_c; do
  if ! runuser -u "$login" -- git config --global --get-all safe.directory | grep -Fxq
"$repo"; then
    runuser -u "$login" -- git config --global --add safe.directory "$repo"
  fi
  install -d -m 700 -o "$login" -g supermarket36 "$run_dir/$login"
  runuser -u "$login" -- git clone "$repo" "$run_dir/$login/work"
done
a="$run_dir/student_a/work"; b="$run_dir/student_b/work"; c="$run_dir/student_c/work"
gita() { runuser -u student_a -- git -C "$a" "$@"; }
gitb() { runuser -u student_b -- git -C "$b" "$@"; }
gitc() { runuser -u student_c -- git -C "$c" "$@"; }
if gita rev-parse --verify HEAD >/dev/null 2>&1; then
  printf 'Существующий репозиторий сохранён. Клонирование под 3 пользователями
успешно.\n'
  gita log --graph --oneline --all
  exit 0
fi
printf '# Автоматическая демонстрация Git Flow\n\nСтатус: подготовка\n' > "$a/README.md"
chown student_a:supermarket36 "$a/README.md"
gita add README.md; gita commit -m 'docs: исходное состояние учебного примера'
gita push origin main; gita switch -c develop; gita push origin develop
gitb fetch origin; gitb switch -c codex/feature/SM-05-review origin/develop
gitc fetch origin; gitc switch -c codex/feature/SM-06-process origin/develop
printf '# Автоматическая демонстрация Git Flow\n\nСтатус: проверка требований\n' >
"$b/README.md"
chown student_b:supermarket36 "$b/README.md"
gitb add README.md; gitb commit -m 'docs: ветка проверки требований'; gitb push origin
HEAD
printf '# Автоматическая демонстрация Git Flow\n\nСтатус: настройка процесса\n' >
"$c/README.md"
chown student_c:supermarket36 "$c/README.md"
gitc add README.md; gitc commit -m 'docs: ветка настройки процесса'; gitc push origin
HEAD
gita fetch origin
gita merge --no-ff origin/codex/feature/SM-05-review -m 'merge: результат проверки'
if gita merge --no-ff origin/codex/feature/SM-06-process -m 'merge: настройка процесса';
then
  printf 'Ожидался учебный конфликт\n' >&2; exit 1
fi
[[ $(gita diff --name-only --diff-filter=U) == README.md ]] || exit 1
```

```
printf '# Автоматическая демонстрация Git Flow\n\nСтатус: требования проверены, процесс
настроен\n' > "$a/README.md"
gita add README.md; gita commit -m 'merge: согласованное разрешение учебного конфликта'
gita switch -c codex/release/1.0.0
gita switch main; gita merge --no-ff codex/release/1.0.0 -m 'release: учебный выпуск'
gita tag -a v1.0.0 -m 'Автоматическая демонстрация Git Flow'
gita switch develop; gita merge --no-ff codex/release/1.0.0 -m 'merge: выпуск в develop'
gita push origin main develop codex/release/1.0.0 --tags
gitb fetch origin; gitc fetch origin
gita diff --check; gita log --graph --oneline --decorate --all
printf 'PASS: 3 clone/push, feature/develop/release/main, merge conflict, tag; автор Lab
automation\n'
printf 'Демонстрационные клоны сохранены в %s\n' "$run_dir"
```

ПРИЛОЖЕНИЕ Б

Протокол проверки системы контроля версий git-checks.txt

```
2026-09-08 Ubuntu 24.04.1 WSL2
Repository: /srv/supermarket36/repository.git
Accounts: student_a student_b student_c gitservice
PASS: three clone/fetch/push operations; feature/develop/release/main; actual merge
conflict; release tag v1.0.0
Initial commit 4e5a646; branches 2769f77 and a050e87; merge de4173d; resolved conflict
9300fc0; release cd1a663.
All demonstration commits authored by Lab automation, not attributed to personal student
work.
```